



Memory Management in C

Storage duration, stack, heap, dynamic allocation, embedded safety, and course synthesis

Embedded Bare-Metal Diploma

DAY 5

6 Hours

```
uint16_t *samples =  
    malloc(8 * sizeof  
    *samples);  
  
if (samples != NULL) {  
    free(samples);  
}
```

Prepared by Mahmoud Morsy

NTI | National Telecommunication Institute



One day turns memory from a hidden detail into a design decision

Session roadmap

00:00

01

Memory map
and
storage
duration

01:10

02

Stack, static
data
and program
memory

02:20

03

Heap
functions and
safe
allocation

03:30

04

Defects,
fragmentatio
n
and
debugging

04:45

05

Embedded
practice
and course
synthesis

Teaching rhythm: explain -> demonstrate -> 5-10 minute task -> review



By the end, students can control lifetime, ownership and RAM use

Learning outcomes

- Identify Flash, EEPROM, SRAM, stack, heap, .data and .bss.
- Distinguish scope, storage duration and object lifetime.
- Explain how function calls consume stack memory.
- Use malloc(), calloc(), realloc() and free() safely.
- Detect leaks, dangling pointers, double-free and bounds errors.
- Choose deterministic memory strategies for ATmega32A applications.

TODAY'S OUTPUT

8

**memory decisions
and solved
exercises**



Memory management protects reliability on a small microcontroller

Why memory matters

01

Finite budget

Code, globals, locals, buffers, heap and stack must fit their physical memories.

02

Valid lifetime

Every object must remain valid for as long as any pointer still uses it.

03

Predictable execution

Embedded control must avoid random failure caused by corruption or fragmentation.

Design rule: decide where data lives, how long it lives, who owns it, and how much space it needs.



C objects live in different memory regions for different reasons

Memory-region overview

FLASH

program instructions
read-only constants

SRAM LOW

.data + .bss
heap grows upward
free space
stack grows downward

SRAM HIGH

Program

Flash

Initialized

.data

Zeroed

.bss

Dynamic

heap

**Stack and heap share SRAM.
Budget both.**



ATmega32A provides only 2 KB of internal SRAM

Memory budget

```
#define FLASH_BYTES    (32UL * 1024UL)
#define EEPROM_BYTES   (1UL * 1024UL)
#define SRAM_BYTES     (2UL * 1024UL)

uint8_t frame[512]; // 25% of SRAM
```

READ THE BUDGET

- 32 KB Flash stores program code.
- 1 KB EEPROM retains data without power.
- 2 KB SRAM holds variables, buffers, heap and stack.
- A 512-byte array consumes one quarter of SRAM.
- Always budget worst-case stack and buffers.

SRAM is the critical limit



Storage duration determines how long an object exists

Automatic, static and allocated storage



Lifetime flow

Automatic objects end at block exit; static objects persist; allocated objects persist until free().



Exercise: separate visibility from lifetime

7-minute practice

- 1 Local int inside a function
- 2 Static counter inside a function
- 3 Block returned by malloc()
- 4 Global configuration object

For each object: state its scope and when its lifetime ends.



Answer: scope controls names; storage duration controls existence

Instructor review

```
void sample(void)
{
    int local = 0;           // block
scope, automatic
    static int calls = 0;    // block
scope, static
    int *p = malloc(4);      // p
automatic; block allocated
```

ASK WHY

**Why can calls
preserve its value
between function
calls?**

**Why does the
allocated block
not end when p
leaves scope?**



The stack stores temporary data for active function calls

Stack frames

```
int readSensor(void)
{
    int raw = 512;
    return raw;
}

int main(void)
{
    int value = readSensor();
}
```

STACK FRAME CONTENT

Item	Stack role
Return address	Where execution continues
Parameters and locals	Temporary call data
Saved registers	CPU state restored later
Lifetime	Until the function returns

The frame ends when the function returns.



Stack overflow usually comes from depth or large local objects

Stack-risk checklist

Cause	Risk	Safer choice
Large local array	Large frame	Static/caller buffer
Deep call chain	Many frames	Flatten the path
Recursion	Unbounded depth	Use a loop
Long ISR nesting	Extra frames	Keep ISRs short
Struct by value	Large copies	Pass const pointer

On ATmega32A, globals, heap and stack share the same 2 KB SRAM budget.



Static objects occupy SRAM throughout execution

[.data and .bss sections](#)

Object	Section	Startup state
int gain = 5;	.data	Copied from Flash
int count;	.bss	Cleared to zero
static int x;	.bss	Cleared to zero
static int y = 2;	.data	Copied from Flash
local automatic	stack	Created on block entry

Initialization data for .data is stored in Flash, then copied into SRAM during startup.



Move read-only lookup data to Flash when SRAM is scarce

Constants and AVR program memory

```
#include <avr/pgmspace.h>

const uint16_t table[4] PROGMEM =
{ 100, 250, 500, 1000 };

uint16_t value =
    pgm_read_word(&table[2]);
```

SRAM

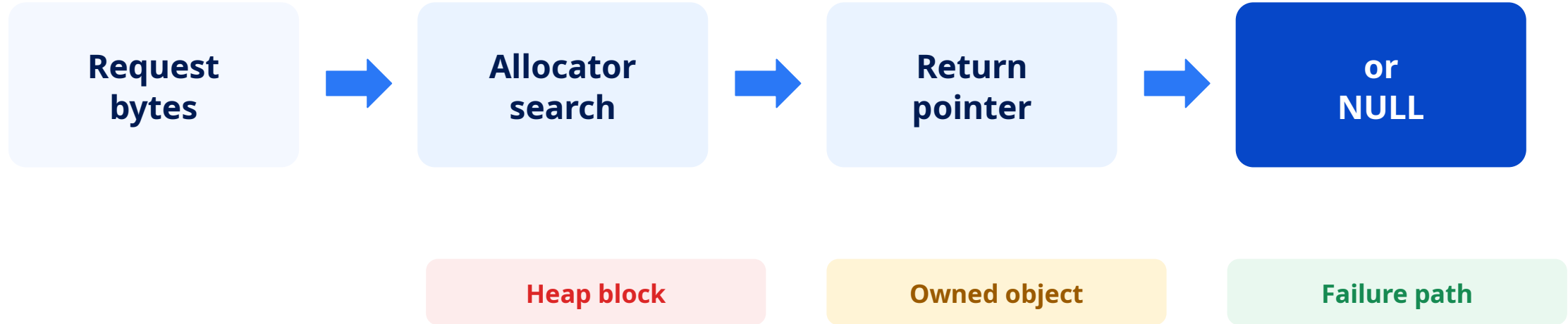
FLASH

ACCESS



The heap supplies variable-size storage at runtime

Dynamic allocation



A successful allocation creates an object whose lifetime continues until `free()` releases it.



malloc() allocates bytes but does not initialize their contents

Allocation contract

```
#include <stdlib.h>
#include <stdint.h>

size_t count = 4;
uint16_t *samples =
    malloc(count * sizeof *samples);

if (samples == NULL)
{
    /* allocation failed */
}
```

TRACE THE REQUEST

count **4 elements**

element **2 bytes**
t

request **8 bytes**



Exercise: allocate a six-reading sensor buffer

10-minute practice

Item	Requirement
Count	6 readings
Element type	uint16_t
Allocation	count * sizeof *samples

REQUIREMENTS

Check NULL

Write six values

Print then free

Work individually - 10 minutes



Answer: size the object, test the pointer, then release it

Instructor review

```
size_t count = 6;
uint16_t *samples = malloc(count * sizeof *samples);

if (samples != NULL)
{
    for (size_t i = 0; i < count; i++)
        samples[i] = (uint16_t)(100 + i);

    free(samples);
    samples = NULL;
}
```

TRACE

Allocate 12 bytes
Check success
Use indices 0..5
Free once
Clear the pointer



calloc() allocates an array and fills its bytes with zero

Zero-filled allocation

```
size_t count = 8;
uint8_t *flags =
    calloc(count, sizeof *flags);

if (flags == NULL)
{
    enterSafeState();
}

/* flags[0..7] begin with zero bytes */
```

WHEN TO USE IT

- Array-like allocations
- Clean initial byte state
- Count x element-size syntax
- NULL check still required
- Release later with free()

Use calloc() when zero-filled elements simplify setup.



malloc() and calloc() differ mainly in request form and initialization

Allocation comparison

MALLOC

One byte count

`malloc(n * sizeof *p)`

CALLOC

Count + size

`calloc(n, sizeof *p)`

BOTH

Return pointer

Check NULL + free

The program must track array capacity after allocation.

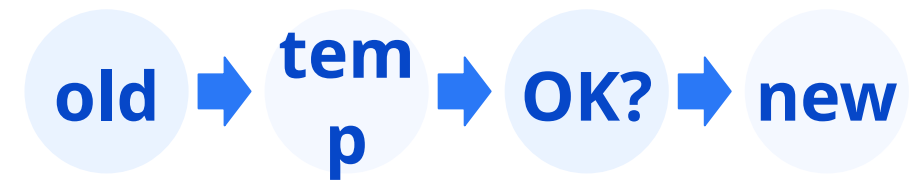


Use a temporary pointer so realloc() failure cannot lose the block

Safe resizing pattern

```
uint16_t *temp =  
    realloc(samples, newCount *  
    sizeof *samples);  
  
if (temp != NULL)  
{  
    samples = temp;  
    count = newCount;  
}
```

Trace both outcomes before assigning.



Success -> assign temp
Failure -> keep old pointer



Exercise: grow a buffer from 10 to 20 samples

10-minute practice

**10 -> 20
samples**

- Use realloc() without overwriting the original pointer until success is confirmed.

**Required: temporary pointer +
NULL test + capacity update**

Work individually - 10 minutes



Answer: commit the resized pointer only after success

Instructor review

```
size_t newCount = 20;
uint16_t *temp =
    realloc(samples, newCount * sizeof
* samples);

if (temp == NULL)
    reportMemoryFault();
else {
    samples = temp;
    count = newCount;
}
```

TRACE

**Success: temp may
move.**

**Failure: samples
stays valid.**

Assign only after success.



free() ends the allocated object's lifetime

Release and ownership

```
uint8_t *buffer = malloc(64);  
  
if (buffer != NULL)  
{  
    useBuffer(buffer);  
  
    free(buffer);  
    buffer = NULL;  
}
```

Function

free(pointer)

Effect

returns block

Does not

erase pointer

Afterward

do not dereference

One clear owner should be responsible for exactly one release.



Exercise: find four lifetime and ownership errors

8-minute code review

```
int *p = malloc(4 * sizeof *p);  
p[4] = 25; free(p); printf("%d", p[0]);  
free(p);
```

- Find the defects and write one correction for each.

Hint: check allocation, bounds, lifetime and release count.

Maximum time - 8 minutes



Answer: validate, stay in bounds, free once and stop using the block

Instructor review

```
int *p = malloc(4 * sizeof *p);

if (p != NULL)
{
    p[3] = 25;
    printf("%d", p[3]);

    free(p);
    p = NULL;
}
```

0

remaining
ownership errors



Memory defects corrupt data long before the program obviously fails

Common failure modes

01

Memory leak

A block becomes unreachable without being released.

02

Dangling pointer

A pointer remains after the object's lifetime has ended.

03

Bounds corruption

A write crosses the allocated object and damages nearby memory.

Also avoid double-free, use-after-free, uninitialized reads and unsafe size calculations.



Each memory defect has a recognizable cause and prevention rule

Defect reference

Defect	Meaning	Prevention
Leak	Lost allocated block	One owner + free
Dangling	Pointer to dead object	End use at lifetime
Double-free	Release twice	Single release path
Out of bounds	Cross object limit	Check index/capacity
Use-after-free	Access released block	Stop use + clear ptr

A pointer value is not proof that its object is still alive.



Fragmentation can prevent allocation even when free bytes remain

Heap fragmentation

```
A = malloc(100);  
B = malloc(100);  
C = malloc(100);  
  
free(B);           // hole: 100 bytes  
D = malloc(140); // may fail to fit the  
hole
```

READ THE PATTERN

- Repeated mixed-size allocation creates holes.
- Total free memory may be split into small regions.
- Long-running systems need predictable timing.
- A heap walk adds variable execution time.
- Fixed-size blocks reduce fragmentation risk.

Fragmentation concerns contiguous free space.



Exercise: choose a deterministic buffer for a sensor logger

10-minute design task

Requirements

- store the latest 16 ADC readings
- run indefinitely
- no runtime allocation failure
- overwrite the oldest sample

Choose the memory strategy and sketch the data structure.

STATIC

POOL

RING



Answer: a static ring buffer gives bounded and reusable storage

Application review

Fixed buffer

Known capacity

```
uint16_t samples[16]  
32 bytes reserved
```

Circular index

Reuse oldest

```
head = (head + 1) % 16
```

No heap

Deterministic

No runtime allocation
or fragmentation

Fixed capacity gives predictable memory demand and execution behavior.



A short checklist catches most memory errors before deployment

Debugging and review

```
if (ptr == NULL)
    enterSafeState();

if (index < capacity)
    ptr[index] = value;

free(ptr);
ptr = NULL;
```

REVIEW EVERY BUFFER

- Is size based on sizeof?
- Is allocation failure handled?
- Are indices inside capacity?
- Is ownership unambiguous?
- Is the block released exactly once?

Inspect the linker map and worst-case stack depth.



Safe memory design starts before allocation

Integrated design workflow

Phase	Required decision
Plan	Capacity, lifetime, owner
Operate	Check failure and bounds
Release	Free once; stop all access

EMBEDDED PRIORITIES

Budget SRAM

Fixed capacity

Failure tests

Use the heap only when variable size justifies memory and timing risk.



Memory-management functions: purpose and critical rules

[Final function reference](#)

```
malloc(bytes)  
    allocate uninitialized bytes  
  
calloc(count, size)  
    allocate zero-filled elements  
  
realloc(ptr, newBytes)  
    resize or move an existing block  
  
free(ptr)  
    release an allocated block
```

MOST IMPORTANT POINTS
Use sizeof *ptr.
Check for NULL.
Use temp with realloc().
One owner; one free.

**Free once. Never access
after free(). Set an inactive
pointer to NULL.**



Five days of C: from syntax to reliable embedded software

Complete course summary

FOUNDATIONS AND CONTROL FLOW

Program structure, build process, variables, types, operators, input/output, conditions, switch, loops and tracing.

DATA, FUNCTIONS AND MODELS

Arrays, strings, pointers, function declarations, parameters, arguments, callbacks, structures and enumerations.

- Explicit control flow
 - Small, reusable functions
- Arrays, pointers, structs and enums
- Bounds, lifetime and ownership
 - Budget Flash, EEPROM and SRAM
- Test failure paths

Next: build ATmega32 peripheral drivers, interrupts and communication protocols.